Experiment No.9

Case Study on Machine Learning Application

Case Study: Disease Prediction Using Machine Learning

Aim : Heart Disease Prediction Using Machine Learning.

---

1. Introduction

Heart disease is one of the leading causes of death worldwide. Early diagnosis can significantly reduce mortality rates. Machine Learning can assist healthcare professionals by predicting the likelihood of heart disease based on patient health records.

This case study demonstrates how Machine Learning is used to predict heart disease risk and support medical decision-making.

---

2. Problem Statement

Develop a Machine Learning model that predicts whether a patient is likely to suffer from heart disease based on medical parameters such as age, blood pressure, cholesterol levels, and heart rate.

---

3. Objectives

- Predict the presence of heart disease.
- Assist doctors in early diagnosis.
- Improve healthcare decision-making.
- Reduce diagnostic errors.

---

4. Dataset Description

The dataset contains patient medical information.

Attribute	Description
Age	Age of patient
Sex	Male/Female
Blood Pressure	Resting blood pressure
Cholesterol	Serum cholesterol level
Heart Rate	Maximum heart rate achieved
Blood Sugar	Fasting blood sugar
Chest Pain Type	Type of chest pain
Target	Heart Disease (Yes/No)

Sample Dataset

Age	BP	Cholesterol	Heart Rate	Disease
45	130	230	150	Yes
38	120	180	170	No
60	140	280	130	Yes
50	125	210	160	No

---

5. Machine Learning Approach

Step 1: Data Collection

Data collected from:

- Hospitals
- Healthcare databases
- Electronic Medical Records (EMR)

Step 2: Data Preprocessing

- Handle missing values

- Remove duplicate records
- Normalize data
- Encode categorical variables

Step 3: Feature Selection

Important features:

- Age
- Blood Pressure
- Cholesterol
- Heart Rate

Step 4: Model Training

Algorithms used:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)

---

6. Mathematical Model

Logistic Regression

The probability of heart disease is:

$$P(Y = 1) = \frac{1}{1+e^{-z}}$$

where

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \cdots + \beta_n X_n$$

Where:

- Y= Disease Prediction
- X_1, X_2, X_3 = Health parameters
- β= Model coefficients

---

7. Implementation Process

Training Phase

1. Load dataset.
2. Split into training and testing data.
3. Train Logistic Regression model.
4. Evaluate performance.

Testing Phase

Input:

- Age = 55
- BP = 145
- Cholesterol = 260
- Heart Rate = 135

Output:

$$P(Heart\ Disease) = 0.87$$

Prediction:

High Risk of Heart Disease

---

8. Results

Algorithm	Accuracy
Logistic Regression	85%
Decision Tree	82%
Random Forest	91%
SVM	88%

Best Model

Random Forest achieved the highest accuracy of **91%**.

---

9. Benefits of Machine Learning

- Early disease detection
- Faster diagnosis
- Reduced healthcare costs
- Better patient management
- Improved treatment planning

---

10. Challenges

- Data privacy concerns
- Missing medical records
- Imbalanced datasets
- Need for large amounts of quality data

---

11. Real-World Impact

Hospitals and healthcare organizations use ML-based systems to:

- Predict heart disease risk
- Monitor patients remotely
- Recommend preventive treatments
- Support clinical decision-making

Examples include:

- IBM Watson Health
- Google Health
- Philips Healthcare

Heart Disease Prediction using Machine Learning

Logistic:

SVM:

Random Forest:

Decision Tree:

Implementation:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = {
   'age' :[45,38,60,50,55,42,65,48,58,36],
   'bp' :[130,120,140,125,145,118,150,128,138,115],
   'cholesterol' :[230,180,280,210,260,190,300,220,250,170],
   'heartrate' :[150,170,130,160,135,175,125,155,140,180],
   'disease' :[1,0,1,0,1,0,1,0,1,0]
}
```

```
df = pd.DataFrame(data)
X = df[['age','bp','cholesterol','heartrate']]
y = df['disease']
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3,random_state=42)
model = SVC()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test,y_pred)
print("Accuracy =",accuracy)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test,y_pred))
# Convert new_patient to a DataFrame with appropriate column names to match training data
new_patient_df = pd.DataFrame([[55,145,260,135]], columns=X.columns)
prediction = model.predict(new_patient_df)
if prediction[0] == 1:
    print("\nHeart Disease Risk: YES")
else:
    print("\nHeart Disease Risk: NO")
```

Output:

1. **Logistic Regression**

```
Accuracy = 1.0

Confusion Matrix:
[[2 0]
 [0 1]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         2
           1       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3


Heart Disease Risk: YES
```

2. Decision Tree Classifier

```
Accuracy = 1.0

Confusion Matrix:
[[2 0]
 [0 1]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         2
           1       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3


Heart Disease Risk: YES
```

3. Random Tree Classifier

```
Accuracy = 1.0

Confusion Matrix:
[[2 0]
 [0 1]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         2
           1       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3


Heart Disease Risk: YES
```

4. Support Vector Machine

```
Accuracy = 1.0

Confusion Matrix:
[[2 0]
 [0 1]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         2
           1       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3


Heart Disease Risk: YES
```

12. Conclusion

Machine Learning provides an effective solution for predicting heart disease using patient medical data. By analyzing health indicators such as age, blood pressure, cholesterol, and heart rate, ML models can identify high-risk patients with high accuracy. Such systems help healthcare professionals make informed decisions, improve patient outcomes, and reduce mortality rates through early intervention.

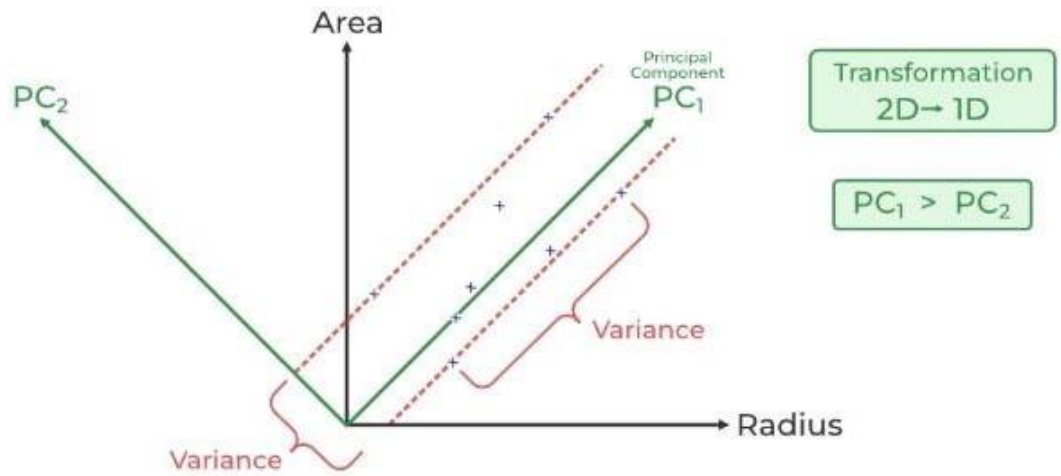
Experiment No. 6

Aim: Implementation of Principle Component Analysis as Dimensionality Reduction Technique**.**

Objective: Able to perform Singular Value Decomposition for obtaining Dimensionality Reduction that yields Principal Component Analysis results.

Theory: Principal Component Analysis is an unsupervised learning algorithm that is used for the dimensionality reduction in machine learning**.** It is a statistical process that converts the observations of correlated features into a set of linearly uncorrelated features with the help of orthogonal transformation. These new transformed features are called the **Principal Components**. It is one of the popular tools that is used for exploratory data analysis and predictive modeling. It is a technique to draw strong patterns from the given dataset by reducing the variances.PCA generally tries to find the lower-dimensional surface to project the high-dimensional data.

PCA works by considering the variance of each attribute because the high attribute shows the good split between the classes, and hence it reduces the dimensionality. Some real-world applications of PCA are ***image processing, movie recommendation system, optimizing the power allocation in various communication channels.*** It is a feature extraction technique, so it contains the important variables and drops the least important variable.



The PCA algorithm is based on some mathematical concepts such as:

- Variance and Covariance
- Eigenvalues and Eigen factors

Some common terms used in PCA algorithm:

- **Dimensionality:** It is the number of features or variables present in the given dataset. More easily, it is the number of columns present in the dataset.
- **Correlation:** It signifies that how strongly two variables are related to each other. Such as if one changes, the other variable also gets changed. The correlation value ranges from -1 to +1. Here, -1 occurs if variables are inversely proportional to each other, and +1 indicates that variables are directly proportional to each other.
- **Orthogonal:** It defines that variables are not correlated to each other, and hence the correlation between the pair of variables is zero.
- **Eigenvectors:** If there is a square matrix M, and a non-zero vector v is given. Then v will be eigenvector if Av is the scalar multiple of v.
- **Covariance Matrix:** A matrix containing the covariance between the pair of variables is called the Covariance Matrix.

Principal Components in PCA

As described above, the transformed new features or the output of PCA are the Principal Components. The number of these PCs are either equal to or less than the original features present in the dataset. Some properties of these principal components are given below:

- The principal component must be the linear combination of the original features.
- These components are orthogonal, i.e., the correlation between a pair of variables is zero.
- The importance of each component decreases when going to 1 to n, it means the 1 PC has the most importance, and n PC will have the least importance.

Steps for PCA algorithm

1. **Getting the dataset**
   Firstly, we need to take the input dataset and divide it into two subparts X and Y, where X is the training set, and Y is the validation set.
2. **Representing data into a structure**
   Now we will represent our dataset into a structure. Such as we will represent the two-dimensional matrix of independent variable X. Here each row corresponds to the data items, and the column corresponds to the Features. The number of columns is the dimensions of the dataset.
3. **Standardizing the data**
   In this step, we will standardize our dataset. Such as in a particular column, the features with high variance are more important compared to the features with lower

variance.
If the importance of features is independent of the variance of the feature, then we will divide each data item in a column with the standard deviation of the column. Here we will name the matrix as Z.

4. **Calculating the Covariance of Z**
   To calculate the covariance of Z, we will take the matrix Z, and will transpose it. After transpose, we will multiply it by Z. The output matrix will be the Covariance matrix of Z.
5. **Calculating the Eigen Values and Eigen Vectors**
   Now we need to calculate the eigenvalues and eigenvectors for the resultant covariance matrix Z. Eigenvectors or the covariance matrix are the directions of the axes with high information. And the coefficients of these eigenvectors are defined as the eigenvalues.
6. **Sorting the Eigen Vectors**
   In this step, we will take all the eigenvalues and will sort them in decreasing order, which means from largest to smallest. And simultaneously sort the eigenvectors accordingly in matrix P of eigenvalues. The resultant matrix will be named as P*.
7. **Calculating the new features Or Principal Components**
   Here we will calculate the new features. To do this, we will multiply the P* matrix to the Z. In the resultant matrix Z*, each observation is the linear combination of original features. Each column of the Z* matrix is independent of each other.
8. **Remove less or unimportant features from the new dataset.**
   The new feature set has occurred, so we will decide here what to keep and what to remove. It means, we will only keep the relevant or important features in the new dataset, and unimportant features will be removed out.

Applications of Principal Component Analysis

- PCA is mainly used as the dimensionality reduction technique in various AI applications such **as computer vision, image compression, etc.**
- It can also be used for finding hidden patterns if data has high dimensions. Some fields where PCA is used are Finance, data mining, Psychology, etc.

Implementation:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

%matplotlib inline
```

```
from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler

from sklearn.datasets import load_breast_cancer data = load_breast_cancer()

data.keys()

print(data['target_names'])  print(data['feature_names'])

df1 = pd.DataFrame(data['data'],columns=data['feature_names'])

scaling=StandardScaler() scaling.fit(df1) scaled_data=scaling.transform(df1)

principal=PCA(n_components=3)

principal.fit(scaled_data)

x=principal.transform(scaled_data)

print(x.shape) principal.components_ plt.figure(figsize=(10,10))

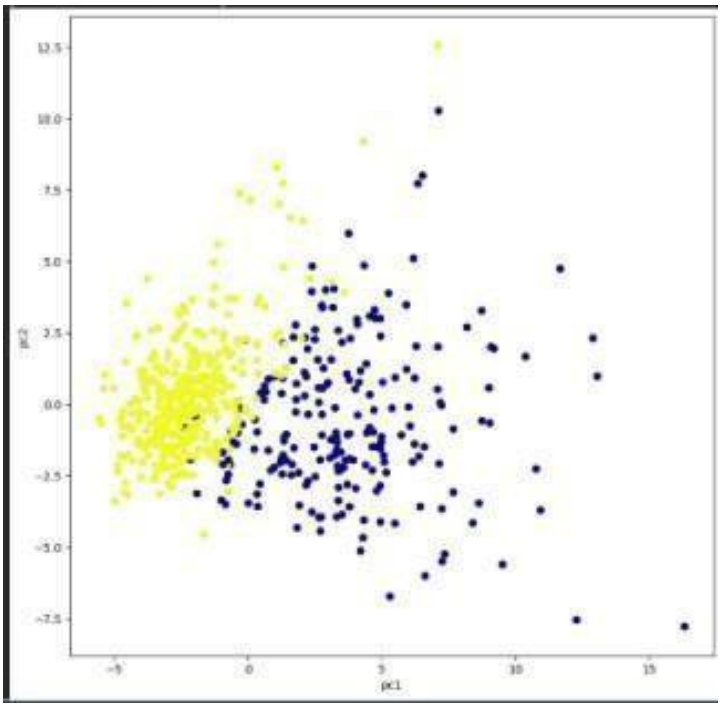
plt.scatter(x[:,0],x[:,1],c=data['target'],cmap='plasma') plt.xlabel('pc1')

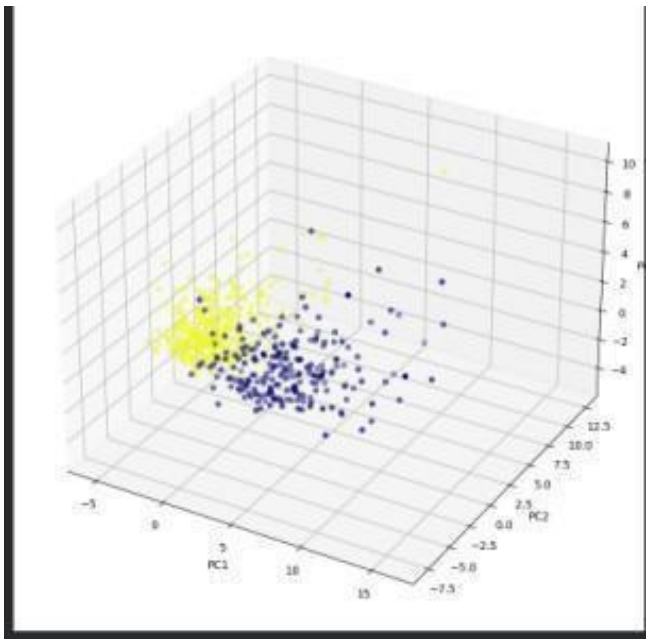
plt.ylabel('pc2')

from mpl_toolkits.mplot3d import Axes3D fig=plt.figure(figsize=(10,10))
axis=fig.add_subplot(111,projection='3d') axis.scatter(x[:,0],x[:,1],x[:,2],c=data['target'],cmap='plasma')
axis.set_xlabel('PC1',fontsize=10) axis.set_ylabel('PC2',fontsize=10) axis.set_zlabel('PC3',fontsize=10)
print(principal.explained_variance_ratio_)
```

Output:

```
['malignant' 'benign']
['mean radius' 'mean texture' 'mean perimeter' 'mean area'
'mean smoothness' 'mean compactness' 'mean concavity' 'mean
concave points' 'mean symmetry' 'mean fractal dimension'
'radius error' 'texture error' 'perimeter error' 'area error'
'smoothness error' 'compactness error' 'concavity error'
'concave points error' 'symmetry error' 'fractal dimension
error' 'worst radius' 'worst texture' 'worst perimeter'
'worst area' 'worst smoothness' 'worst compactness' 'worst
concavity' 'worst concave points' 'worst symmetry' 'worst
fractal dimension'] (569, 3)
[0.44272026 0.18971182 0.09393163]
```





Conclusion: PCA (Principal Component Analysis) is a statistical technique used to reduce the dimensionality of a dataset while retaining most of the important information. It transforms a large number of correlated variables into a smaller set of uncorrelated variables called principal components. PCA is commonly used to simplify complex data, identify important patterns and relationships, reduce redundancy, and visualize high-dimensional data. It is also useful in machine learning and data analysis because reducing the number of features can make models faster and easier to interpret.

ML Experiment No. 1

Aim: To implement Linear Regression for predicting continuous numerical values and analyze the relationship between independent and dependent variables.

Objective: To implement Linear Regression in order to build a model that studies the relationship between an independent and dependent variable. The model will be evaluated by using least square regression method where RMSE and R-squared will be the model evaluation parameters..

Theory:

Linear Regression is a fundamental supervised learning algorithm used to model the relationship between a dependent variable and one or more independent variables. It predicts continuous values by fitting a straight line that best represents the data.

- It assumes that there is a linear relationship between the input and output
- Uses a best-fit line to make predictions
- Commonly used in forecasting, trend analysis, and predictive modelling

The least-squares method is a crucial statistical method that is practiced to find a regression line or a best-fit line for the given pattern. This method is described by an equation with specific parameters. The method of least squares is generously used in evaluation and regression. In regression analysis, this method is said to be a standard approach for the approximation of sets of equations having more equations than the number of unknowns. The method of least squares actually defines the solution for the minimization of the sum of squares of deviations or the errors in the result of each equation. Find the formula for sum of squares of errors, which help to find the variation in observed data. The least-squares method is often applied in data fitting.

Least Squares Regression Example
Assume a problem where you have to predict the weight of the person given his height.
Here weight is the dependent variable and height is independent variable.

Height of the person in cms (x)	Weight of the person in kgs (y)
160	72
171	76
182	77
180	83
154	76

Let us use the concept of least squares regression to find the line of best fit for the above data.

Step 1: Calculate the slope 'm' by using the following formula:

$$m = \frac{n\sum xy - (\Sigma x)(\Sigma y)}{n\Sigma x^2 - (\Sigma x)^2}$$

Step 2: Compute the y-intercept value

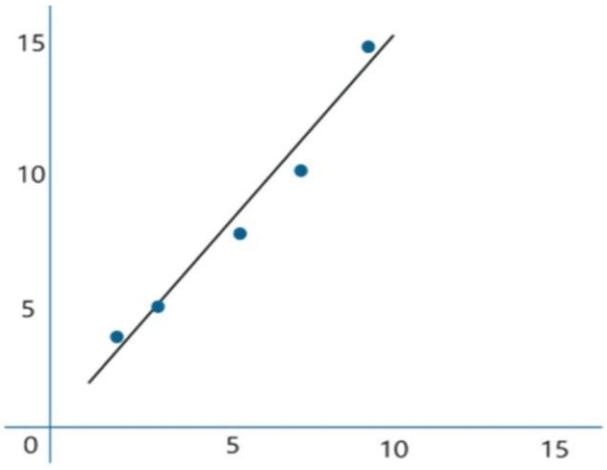
$$c = y - mx$$

\

Step 3: Substitute the values in the final equation

$$y = mx + c$$

Let's construct a graph that represents the **y=mx + c** line of best fit:



Step 1: Import the required libraries

Step 2: Import the data set

Step 3: Assigning 'X' as independent variable and 'Y' as dependent variable

Step 4: Calculate the values of the slope and y-intercept

Step 5: Plotting the line of best fit

Step 6: Model Evaluation

Code:

A)

```
import numpy as np

import matplotlib.pyplot as plt

#Dataset

x= np.array([160,171,182,180,154])

y=np.array([72,76,77,83,76])

# Calculate Mean

x_mean = np.mean(x)

y_mean = np.mean(y)

# Number of observations

n = len(x)

#Calculate slope(b1)

numerator=0

denominator=0

for i in range(n):
```

```
    numerator += (x[i]-x_mean)*(y[i]-y_mean)
    denominator += (x[i]-x_mean)**2
b1=numerator/denominator
b0=y_mean-(b1*x_mean)
# Calculate intercept
# b0 is already calculated above, removing redundant line.
print("slope(b1)=",b1)
print("intercept(b0)=",b0)
# Predicition Function
def predict(x_val):
  return b0+b1*x_val
#Predict weight for height=175 cm
height=175
predicted_weight=predict(height)
print("Predicted weight for height",height,"cm =",predicted_weight)
#Predicted values for all points
y_pred = predict(x)
#Plot
plt.scatter(x,y,color='blue',label='Actual Data')
plt.plot(x,y_pred,color='red',linewidth=2,label='Regression Line')
plt.xlabel('Height(cm)')
plt.ylabel('Weight(kg)')
plt.title("Linear Regression from scratch")
plt.legend()
```

```
plt.grid(True)
plt.show()
```

B)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
#Dataset
data = {
    'Area':[1000,1200,1500,1800,2000,2200,2500,2800,3000,3500],
    'Price':[20,25,30,35,40,45,50,55,60,65]
}
df=pd.DataFrame(data)
print("Dataset")
print(df)
#Feature and Target
x=df['Area'].values.reshape(-1, 1)
y=df['Price']
#Train test split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
#Model
```

```
model=LinearRegression()

model.fit(x_train,y_train)

#Prediction

y_pred=model.predict(x_test)

#Evaluation

mse=mean_squared_error(y_test,y_pred)

r2=r2_score(y_test,y_pred)

print("\nMean Squared Error:",mse)

print("R2 Score=",r2)

#Coefficients

print("Intercept:",model.intercept_)

print("\nSlope:",model.coef_[0])

#Graph

plt.scatter(df['Area'],df['Price'],color='blue')

plt.plot(x_test,y_pred,color='red')

plt.xlabel("Area")

plt.ylabel("Price")

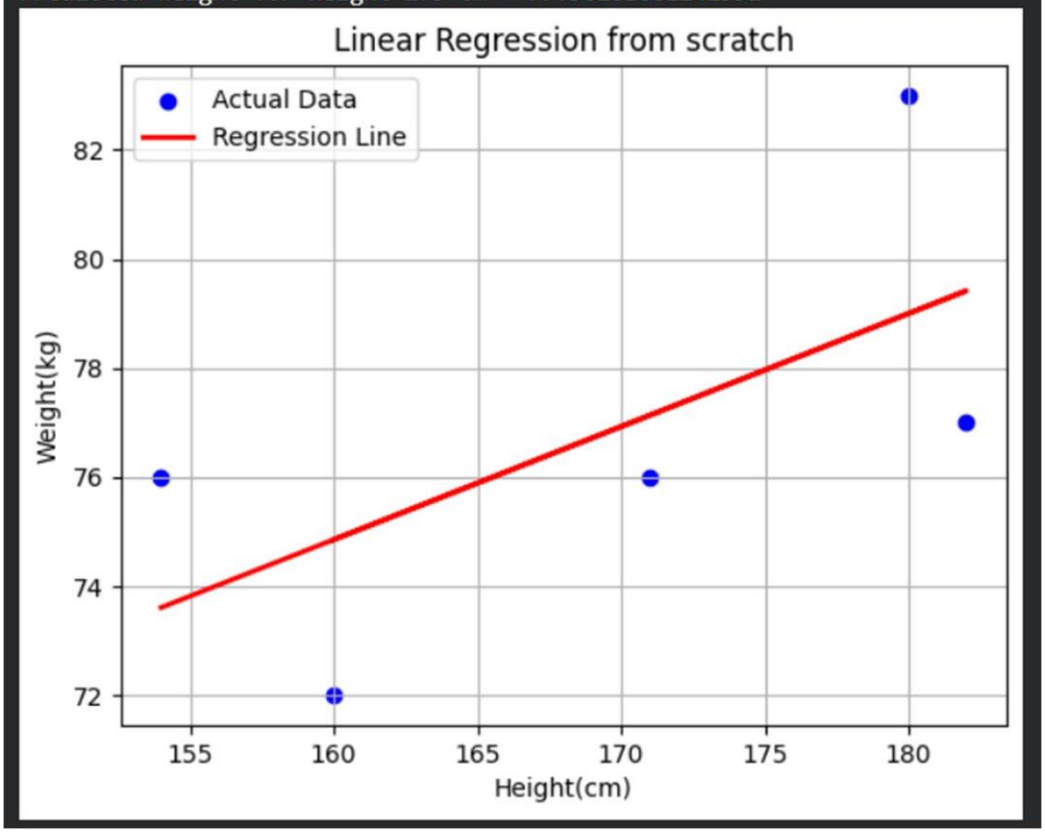
plt.title("Linear Regression")

plt.show()
```

Output:

A)

```
slope(b1)= 0.2076101468624833
intercept(b0)= 41.63084112149532
Predicted weight for height 175 cm = 77.96261682242991
```

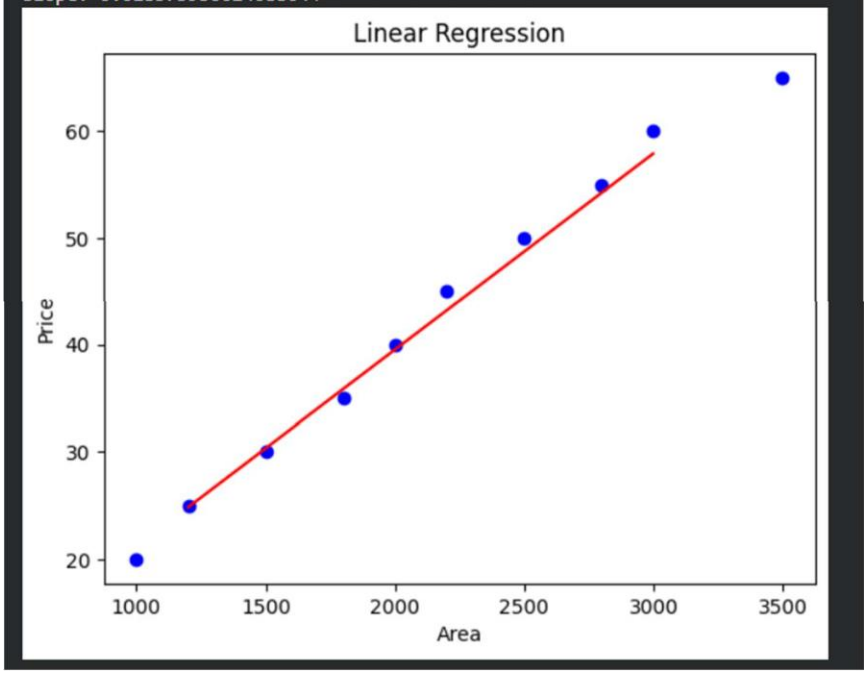


B)

```
Dataset
   Area  Price
0  1000     20
1  1200     25
2  1500     30
3  1800     35
4  2000     40
5  2200     45
6  2500     50
7  2800     55
8  3000     60
9  3500     65

Mean Squared Error: 2.2470064782282693
R2 Score= 0.9926628359894587
Intercept: 2.766363369533309

Slope: 0.018373936014088644
```



Conclusion:

Linear Regression was successfully implemented manually by calculating the slope and intercept. The experiment helped understand the mathematical concepts behind regression and prediction.

Linear Regression was successfully implemented using Scikit-Learn to predict house prices. The model showed good prediction accuracy and demonstrated the practical application of regression analysis.

ML Experiment No. 2

Aim: Implementation of Logistic Regression Algorithm.

Objective: Able to perform various feature engineering tasks, apply logistic regression on the given dataset and maximize the accuracy.

Theory:

Logistic Regression was used in biological sciences in early twentieth century. It was then used in many social science applications. Logistic Regression is used when the dependent variable(target) is categorical and is binary in nature. In order to perform binary classification, the logistic regression techniques makes use of Sigmoid function.

What is Logistics Regression:

Logistic regression is a statistical method used for binary classification problems, where the outcome or dependent variable has two possible values, often represented as 0 and 1. Unlike linear regression, which predicts continuous values, logistic regression predicts the probability that a given input point belongs to a specific class.

Key Concepts:

1. **Sigmoid Function:** Logistic regression uses the sigmoid function to map predicted values to probabilities. The sigmoid function, σ(z)\sigma(z)σ(z), is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$.

Binary Outcome: The dependent variable yyy is binary, taking values of 0 or 1. The logistic function outputs probabilities between 0 and 1, which can then be mapped to these binary outcomes using a threshold (typically 0.5).

Log-Odds and Odds Ratio: Logistic regression models the log-odds of the probability of the outcome:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

where p is the probability of the outcome being 1.

Applications:

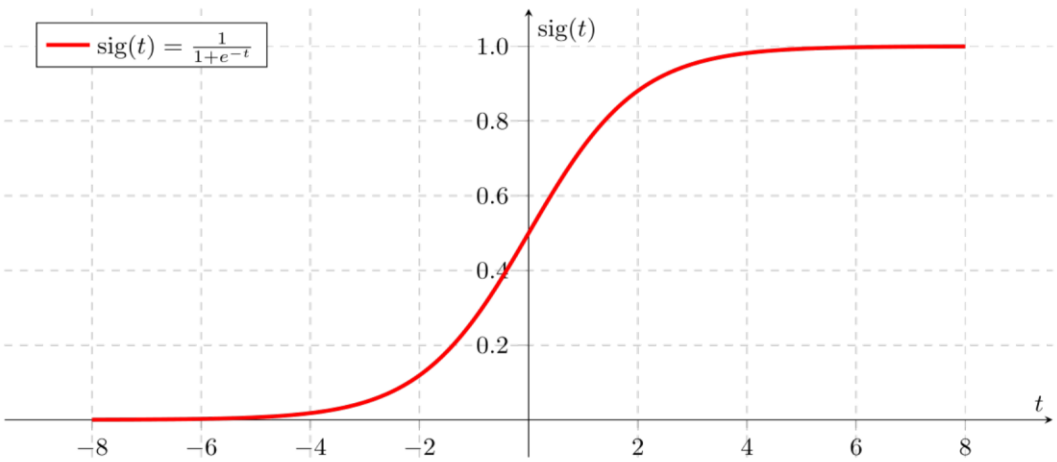
- **Medical:** Predicting the presence or absence of a disease.
- **Finance:** Predicting the likelihood of a customer defaulting on a loan.
- **Marketing:** Determining whether a customer will buy a product.
- **Binary Classification Problems:** Any problem where the outcome is one of two categories.

For example,

To predict whether an email is spam (1) or (0)

Whether the tumor is malignant (1) or not (0)

Consider a scenario where we need to classify whether an email is spam or not. If we use linear regression for this problem, there is a need for setting up a threshold based on which classification can be done. Say if the actual class is malignant, predicted continuous value 0.4 and the threshold value is 0.5, the data point will be classified as not malignant which can lead to serious consequence in real time.



From this example, it can be inferred that linear regression is not suitable for classificatior problem. Linear regression is unbounded, and this brings logistic regression into picture. Thei value strictly ranges from 0 to 1.

Implementation:

Program 1:

```
import numpy as np

X = np.array([.50, 1.50, 2.00, 4.25, 3.25, 5.50], ndmin=2).reshape((6,1))

y = np.array([0, 0, 0, 1, 1, 1])

x_mean = np.mean(X)

y_mean = np.mean(y)

n = len(X)

class LogisticRegression:

    b0 = 0

    b1 = 0

    def fit(self, X, y):

        upward_function = 0

        downward_function = 0


        for i in range(n):

            upward_function += (X[i] - x_mean) * (y[i] - y_mean)

            downward_function += (X[i] - x_mean) ** 2
```

```
        self.b1 = upward_function / downward_function

        self.b0 = y_mean - (self.b1 * x_mean)


return self.b0, self.b1

    def predict(self, Xi):

        z = self.b0 + (self.b1 * Xi)

        sigmoid = 1 / (1 + np.exp(-z))

        if sigmoid >= 0.5:

            y_pred = 1

        else:

            y_pred = 0

        return sigmoid, y_pred

model = LogisticRegression()

print(model.fit(X, y))

model.predict(2.7)
```

Output:

```
(array([-0.2409201]), array([0.26150121]))
(array([0.6142312]), 1)
```

Program 2:

```
# Implementation:

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix

# Dataset

data = {

    'Marks': [25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80],

    'Result': [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1]

}

# Create DataFrame

df = pd.DataFrame(data)

print("Dataset")

print(df)

# Features and Target

X = df[['Marks']]

y = df['Result']

# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#create model

model = LogisticRegression()

model.fit(X_train, y_train)

# Predict test data

y_pred = model.predict(X_test)

print("\nPredicted Values:")

print(y_pred)

# Accuracy

acc = accuracy_score(y_test, y_pred)

print("\nAccuracy:", acc)

# Confusion matrix

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")

print(cm)

# Predict New Student

new_marks = [[58]]

prediction = model.predict(new_marks)

print("\nPrediction for Student with Marks =", new_marks[0][0])

if prediction[0] == 1:

    print("Pass")

else:
```

```
    print("Fail")

plt.figure(figsize=(8, 5))

plt.scatter(

    X[y == 0],

    y[y == 0],

    color='red',

    marker='o',

    s=100,

    label='Fail'

)

plt.scatter(

X[y == 1],

    y[y == 1],

    color='green',

    marker='s',

    s=100,

    label='Pass'

)

X_range = np.linspace(20, 85, 500).reshape(-1, 1)

y_prob = model.predict_proba(X_range)[:, 1]

plt.plot(X_range, y_prob, color='blue', linewidth=3, label='sigmoid curve')

plt.axhline(
```

```
    y=0.5,
    color='black',
    linestyle='--',
    label='Decision Boundary'
)
plt.xlabel('Marks')
plt.ylabel('Probability of Passing')
plt.title('Logistic Regression - Student Pass/Fail Prediction')
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
    Dataset
        Marks  Result
0          25       0
1          30       0
2          35       0
3          40       0
4          45       0
5          50       1
6          55       1
7          60       1
8          65       1
9          70       1
10         75       1
11         80       1

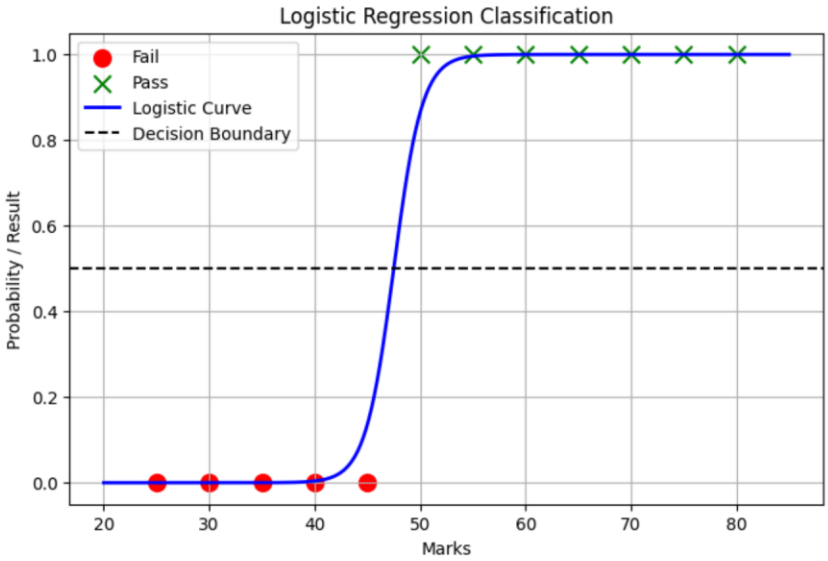
Predicted Values:
[1 1 0]

Accuracy = 1.0

Confusion Matrix:
[[1 0]
 [0 2]]

Prediction for Student with Marks = 58
PASS
```

...



Conclusion: Thus, Logistic Regression was successfully implemented for student Pass/Fail prediction. The model achieved high accuracy and correctly classified students based on their marks using a sigmoid function and decision boundary.

ML Experiment No. 4

Aim: To implement Multivariate Linear Regression using Python and predict the target variable based on multiple independent variables.

Objectives: Multivariate Linear Regression is an extension of simple linear regression where more than one independent variable is used to predict a dependent variable.

The mathematical model is:

$$Y = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + \cdots + b_nX_n$$

Where:

- Y= Dependent Variable
- $X_1, X_2, X_3, \ldots$= Independent Variables
- b_0= Intercept
- $b_1, b_2, b_3, \ldots$= Regression Coefficients

General Equation

The multivariate linear regression model is:

$$Y = \beta_0 + \beta_1X_1 + \beta_2X_2 + \beta_3X_3$$

where:

- Y= Marks
- X_1= Study Hours
- X_2= Attendance
- X_3= Assignments
- β_0= Intercept
- $\beta_1, \beta_2, \beta_3$= Coefficients

Implementation:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score



data = {
    'Study_Hours': [2,3,4,5,6,7,8,9,10,11],
    'Attendance' : [60,65,70,75,80,85,90,92,95,98],
    'Assignements' : [3,4,5,5,6,7,8,8,9,10],
    'Marks' : [40,45,50,55,60,65,70,78,85,90]
}
df = pd.DataFrame(data)
# independent variables
X = df[['Study_Hours','Attendance','Assignements']]
# dependent variable
y = df['Marks']
#split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#Create model
model = LinearRegression()
#train model
model.fit(X_train, y_train)
#make predictions
y_pred = model.predict(X_test)
# results
print("Regression coefficients:")
print(model.coef_)
print("\nIntercept:")
print(model.intercept_)
```

```
print("\nActual values:")
print(y_test.values)
print("\nPredicted values:")
print(y_pred)
#Evaluation
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("\nMean Squared Error:", mse)
print("R-squared:", r2)
```

Output :

```
Regression coefficients:
[ 8.97329377 -0.70029674 -0.56083086]

Intercept:
65.71810089020771

Actual values:
[85 45]

Predicted values:
[83.87537092 44.87537092]

Mean Squared Error: 0.6401614877299424
R-squared: 0.9983995962806751
```

Conclusion :

Multivariate Linear Regression predicts a dependent variable using multiple predictors.The Multivariate Linear Regression model was successfully implemented using multiple independent variables (Study Hours, Attendance, and Assignments) to predict student marks.

Experiment No. 5

Aim: Implementation of Support Vector Machine Algorithm.

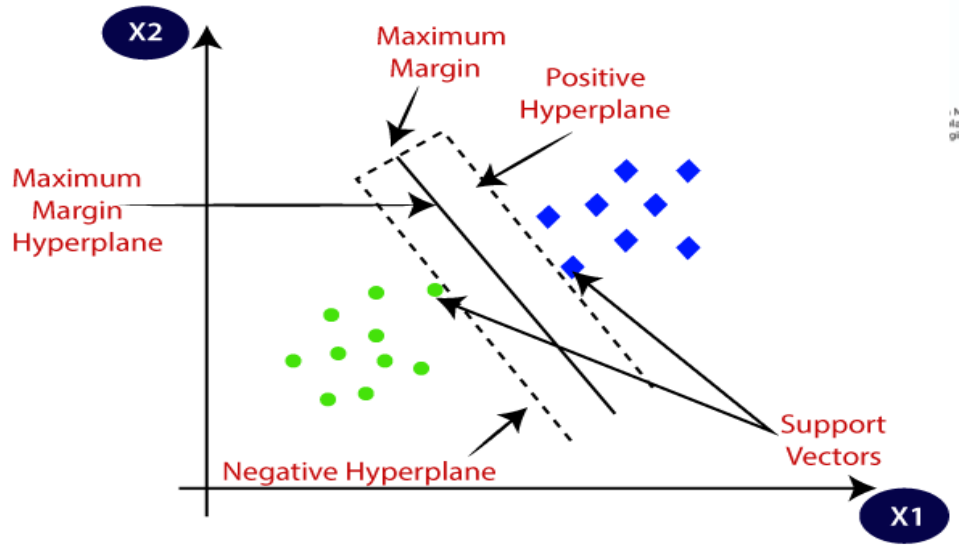
Objective: Ability to perform various feature engineering tasks, apply Support Vector Machine and create the Confusion Matrix.

Theory:

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



SVM implementation in Python

Predict if cancer is Benign or malignant. Using historical data about patients diagnosed with cancer enables doctors to differentiate malignant cases and benign ones are given independent attributes.

Steps

- Load the breast cancer dataset from sklearn.datasets
- Separate input features and target variables.
- Build and train the SVM classifiers using RBF kernel.
- Plot the scatter plot of the input features.
- Plot the decision boundary.
- Plot the decision boundary

Implementation:

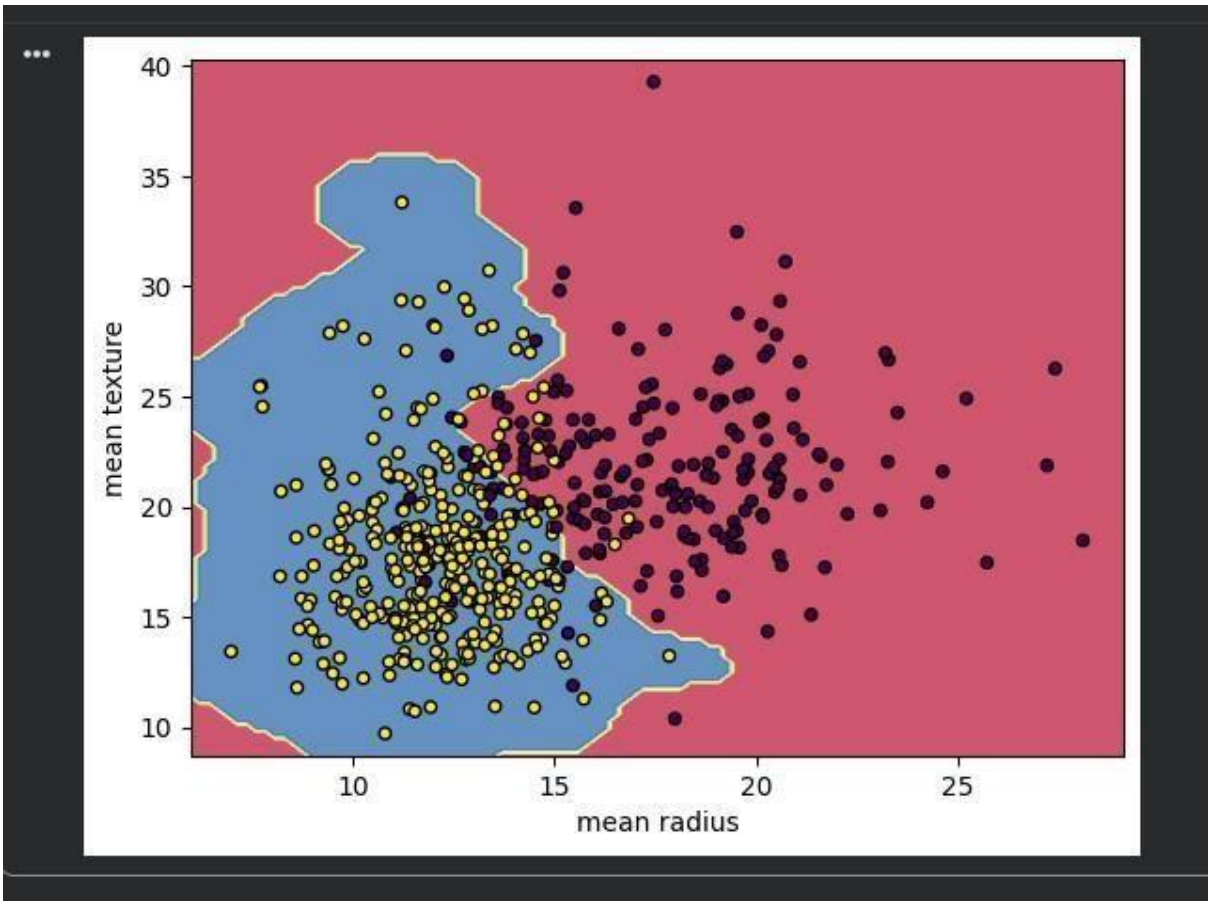
Program for SVM:

```
from sklearn.datasets import load_breast_cancer
import matplotlib.pyplot as plt
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.svm import SVC

cancer = load_breast_cancer()
X = cancer.data[:, :2]
y = cancer.target
svm = SVC(kernel="rbf", gamma=0.5, C=1.0)

svm.fit(X, y)
DecisionBoundaryDisplay.from_estimator(
   svm,
   X,
   response_method="predict",
   cmap=plt.cm.Spectral,
   alpha=0.8,
   xlabel=cancer.feature_names[0],
   ylabel=cancer.feature_names[1],
)
plt.scatter(X[:, 0], X[:, 1], c=y, s=20, edgecolors="k")
plt.show()
```

Output:



Conclusion: We have Successfully Implemented Support Vector Machine Algorithm..

ML Experiment No.3

Implementation of Ensemble Learning (Bagging and Boosting)

Aim: To implement Ensemble Learning techniques using Bagging (Random Forest) and Boosting (AdaBoost) and compare their performance.

Theory :

Ensemble Learning combines multiple machine learning models to improve prediction accuracy.

1. Bagging (Bootstrap Aggregating)

- Creates multiple subsets of training data using random sampling with replacement.
- Trains a separate model on each subset.
- Final prediction is obtained by majority voting (classification) or averaging (regression).
- Example: Random Forest.

2. Boosting

- Models are trained sequentially.
- Each new model focuses on correcting errors made by previous models.
- Final prediction is a weighted combination of all models.
- Example: AdaBoost.

Implementation:

```
# Ensemble Learning using Bagging and Boosting
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

print(" Bagging(Random Forest)")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

ab=AdaBoostClassifier(n_estimators=50, learning_rate=1.0, random_state=42)
ab.fit(X_train, y_train)
ab_pred = ab.predict(X_test)
print("\n Boosting(AdaBoost)")
print("Accuracy:", accuracy_score(y_test, ab_pred))
print("Classification Report:\n", classification_report(y_test, ab_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, ab_pred))
```

Output :

```
bagging
1.0
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45

boosting
1.0
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Result:

The Bagging (Random Forest) and Boosting (AdaBoost) ensemble methods were successfully implemented on the Iris dataset. Both methods achieved high classification accuracy, demonstrating the effectiveness of ensemble learning techniques.

Conclusion:

- Ensemble Learning improves prediction performance by combining multiple models.
- Bagging reduces variance and helps prevent overfitting by training models independently.
- Boosting reduces bias by sequentially correcting previous errors.
- In this experiment, Random Forest (Bagging) achieved slightly better accuracy than AdaBoost (Boosting) on the Iris dataset.
- Ensemble methods generally perform better than a single classifier and are widely used in real-world machine learning applications.

Experiment No.7

Aim: To implement **Graph-Based Clustering (Spectral Clustering)** and group similar data points into clusters using graph theory concepts.

Theory

Graph-based clustering represents data as a graph:

- **Nodes (Vertices):** Data points
- **Edges:** Similarity between data points
- **Weights:** Strength of similarity

The objective is to partition the graph into clusters such that:

- Similar nodes belong to the same cluster.
- Dissimilar nodes belong to different clusters.

Similarity Matrix

For two points x_i and x_j:

$$W_{ij} = e^{-\frac{||x_i - x_j||^2}{2\sigma^2}}$$

where:

- W_{ij} = Similarity between nodes
- σ = Scaling parameter

Degree Matrix

$$D_{ii} = \sum_j W_{ij}$$

Graph Laplacian

$$L = D - W$$

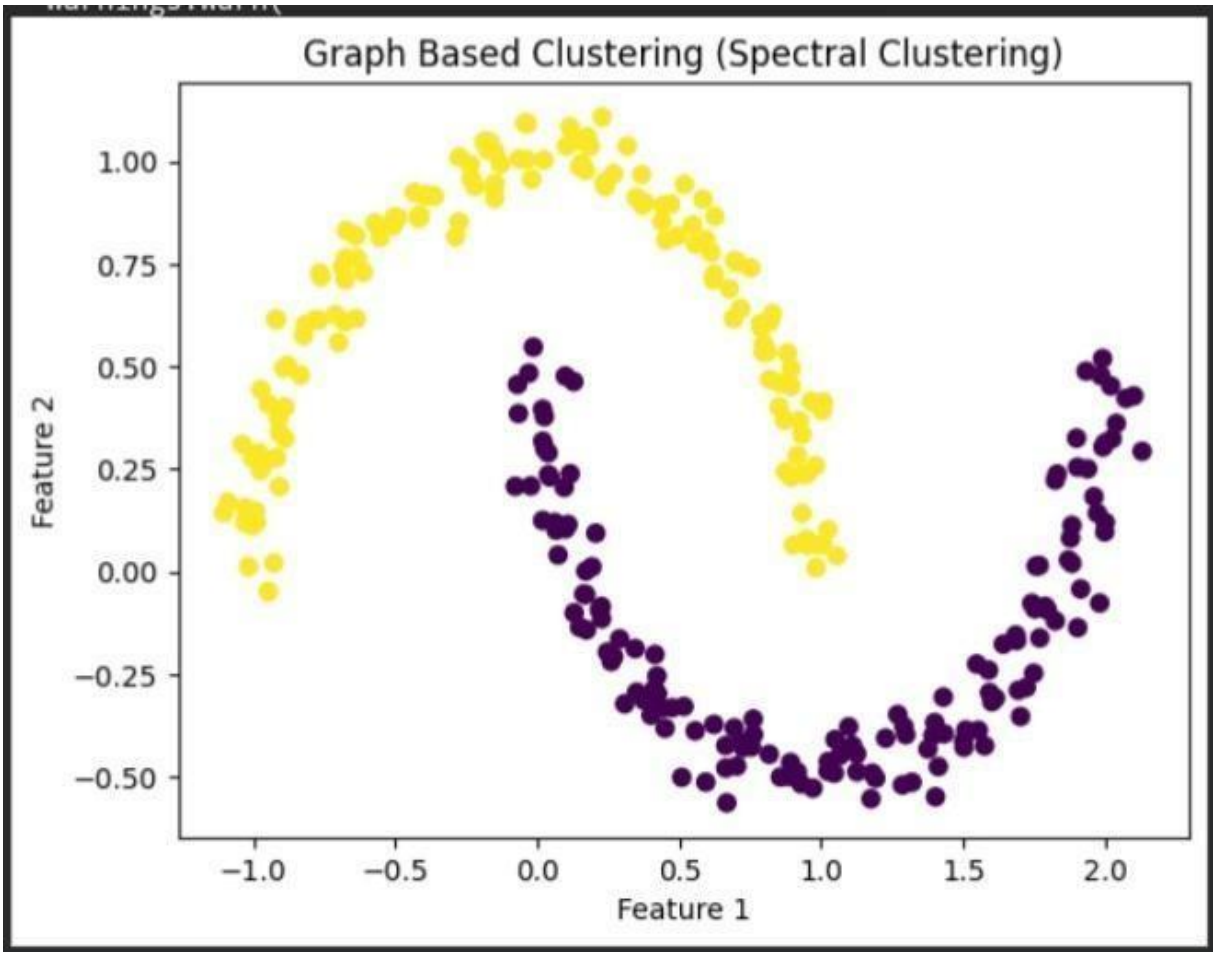
The eigenvectors of the Laplacian matrix are used to identify clusters.

Implementation:

Program:

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.cluster import SpectralClustering
# Generate sample dataset
X, y = make_moons(n_samples=300, noise=0.05, random_state=42)
# Apply Spectral Clustering
spectral = SpectralClustering(
    n_clusters=2,
    affinity='nearest_neighbors',
    random_state=42
)
labels = spectral.fit_predict(X)
# Plot Clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title("Graph Based Clustering (Spectral Clustering)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

Output:



Conclusion :

Graph-Based Clustering using Spectral Clustering was successfully implemented. The algorithm effectively identified non-linear clusters by utilizing graph connectivity and similarity relationships between data points.

Experiment No.8

Aim: To implement DBSCAN (Density-Based Spatial Clustering of Applications with Noise) Clustering and identify clusters and outliers in a dataset.

Objectives:

DBSCAN is a density-based clustering algorithm that groups together points that are closely packed and marks points in low-density regions as outliers (noise).

Parameters

1. Eps (ε):
    - Maximum distance between two points to be considered neighbors.
2. MinPts:
    - Minimum number of neighboring points required to form a dense region.

Types of Points

- Core Point: Has at least MinPts neighbors within ε distance.
- Border Point: Has fewer than MinPts neighbors but lies within ε of a core point.
- Noise Point: Neither a core point nor a border point.

Mathematical Definition

The Euclidean distance between two points $x_i$and $x_j$is:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^{n} (x_{ik} - x_{jk})^2}$$

A point $p$belongs to the ε-neighborhood of point $q$if:

$$d(p, q) \leq \varepsilon$$

Implementation :

Program :

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import DBSCAN

from sklearn.datasets import make_moons


# Generate sample dataset

X, y = make_moons(n_samples=300, noise=0.05, random_state=42)


# Apply DBSCAN

dbscan = DBSCAN(eps=0.2, min_samples=5)

labels = dbscan.fit_predict(X)


# Number of clusters (excluding noise label -1)

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

print("Number of Clusters:", n_clusters)

print("Cluster Labels:", set(labels))


# Plot clusters

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')

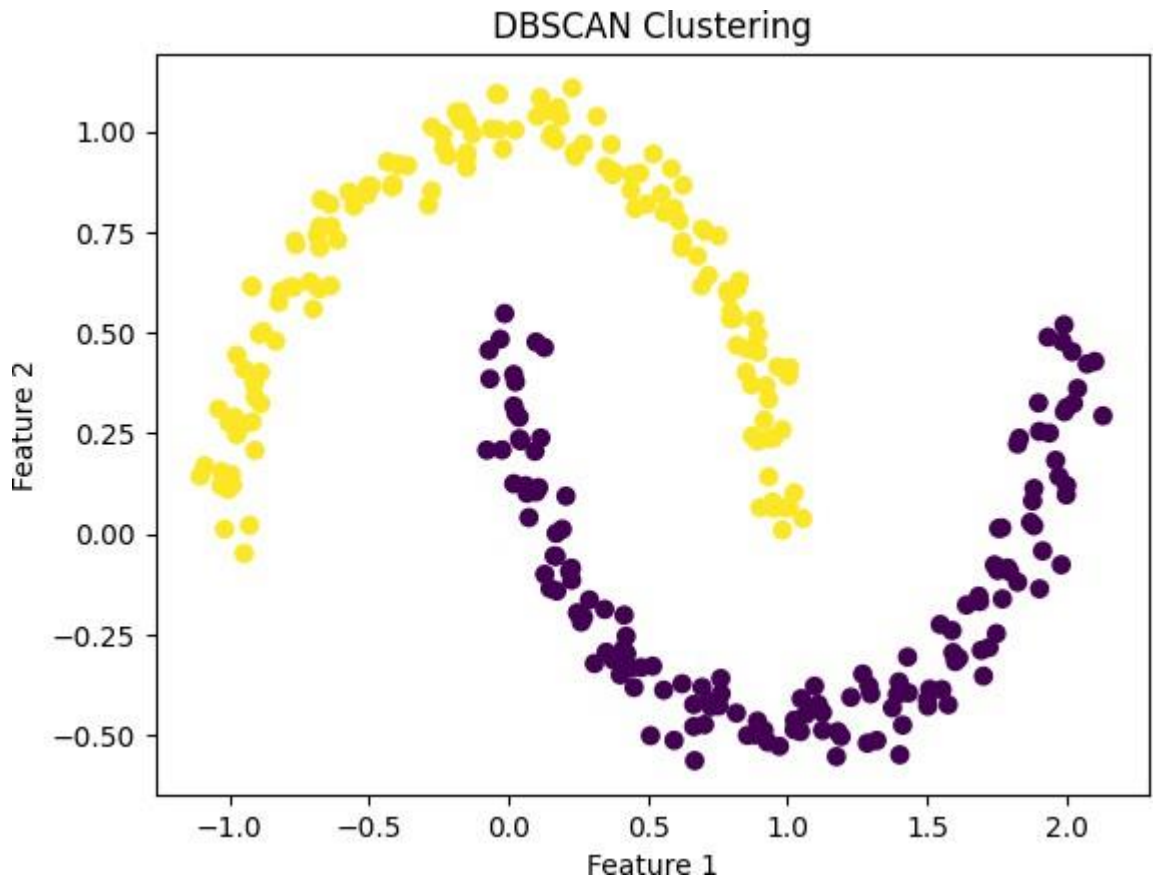
plt.title("DBSCAN Clustering")
```

```
plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.show()
```

Output :



Conclusion :

The DBSCAN clustering algorithm was successfully implemented. The dataset was grouped into clusters based on density, and noise points were automatically detected.